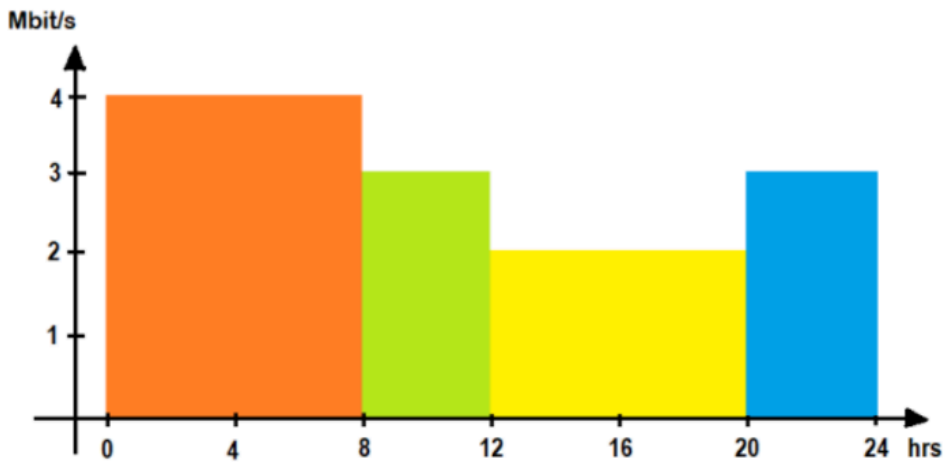
ET Revision Jan25

Q1. The bandwidth variation of a certain network for an entire day is given in the figure below. For the first 12 hours, the total bandwidth was equally divided among the two devices. In the next twelve hours, the number of devices connected to the network got increased by 2 with all the devices sharing the bandwidth equally. What is the amount of data (in GigaBytes) received by the device that was connected throughout the day? (Take 1 GB = 1000 MB) [MCQ: 4.5 points]



- ○ 8.1
- ○ 16.2
- √ 13.05
- ○ 4.05

Solution: BW Mb

BW = no. of req. X size of req, BW= Mb/s Data/time, BW x time=data

$$= \frac{(4Mbps/2users * 8\ hr)+(3Mbps/2users * 4hr)+(2/4Mbps * 8hr)+(3/4Mbps *4hr)) *3600(s)}{1000(G)*8(byte)}$$

$$= \frac{(16+6+4+3)*3600}{1000*8}$$

= 13.05 GB

If asked whole data, no need to divide by users,

Q2. Consider the following flask application:

[MCQ: 4 Points]

```
from flask import Flask, request
app = Flask(__name__)

@app.route('/get_value')
def get_value():
    val1 = request.args.get("val1")
    return "The value is "+ val1

app.run(port = 5000, debug = True)
```

Which of the following route(s) will return `The value is 10` in the browser?

A. `http://127.0.0.1:5000?val1=10`
B. `http://127.0.0.1:5000/get_value?10`
C. `http://127.0.0.1:5000/get_value?val1=10`
D. `http://127.0.0.1:5000/get_value?val1=10&val2=10`

Answer: C,D

A. Not found
B. `TypeError: can only concatenate str (not "NoneType") to str`

Query parameters: ?pass values in the url> request.args
sys.argv>command line arguments> pass in command line
Giving input through forms> request.form.get

Q3. Consider the following HTML document.

[MCQ: 3 points]

```
<!DOCTYPE html>
<html lang="en">
<head>
    <style>
        p {
            background-color: yellow ;
            color: red;
            display: inline-block;
        }
        #id {
            border: 2px solid purple;
            color:blue ;
        }
        .class {
            background-color: aqua;
            color: red;
        }
    </style>
</head>
<body>
    <p class="class" id="id">HTML</p>
    <p class="class" >HTML</p>
    <p>HTML</p>
</body>
</html>
```

How will the browser render above HTML file?

Styling: inline> internal> external
Selectors: !important>id>class>element

A.

HTML

HTML

HTML

B.

HTML HTML HTML

C.

HTML HTML HTML

D.

HTML HTML HTML

Answer: C

Q4: Consider the following Python code snippet.

[MCQ : 3 points]
(Concept: Python string and Jinja2 templates)

```
from string import Template as T1
from jinja2 import Template as T2
info = {'girl1': 'Mary', 'girl2': 'Samantha',
'girl3': 'girl2', 'girl4': 'girl1',
'city1':'Moscow', 'city2':'city1'}

t1 = T1("{{$girl3}} and {{$girl4}} arrived at the {{$city2}} station early
and waited for the bus.")
out1 = t1.substitute(info)
out2 = T2(out1)
print(out2.render(info))
```

What will be the output on the terminal?

A. girl2 and girl1 arrived at the city1 station early and waited for the bus.
B. Samantha and Mary arrived at the Moscow station early and waited for the bus.
C. Mary and Samantha arrived at the Moscow station early and waited for the bus.
D. Error

Answer: B

String: $, substitute(), throw keyerror if value is not passed but safe_substitute()> print var
Jinja: {{}}, render(), blank

Q5. Which of the following flask view functions will return a 404 error for the URL: `http://127.0.0.1:5000/details/1001/Michael` ?

[MCQ: 3 points]

A.

```
@app.route('/details/<int:id>/<string:name>')
def show(id, name):
    details = {'student_id': id, 'student_name': name}
    return details
```

B.

```
@app.route('/details/<string:id>/<string:name>')
def show(id, name):
    details = {'student_id': id, 'student_name': name}
    return details
```

C.

```
@app.route('/details/<id>/<name>')
def show(id, name):
    details = {'student_id': id, 'student_name': name}
    return details
```

D. None of the above.

Answer: D

Bydefault string converter

```
@app.route("/details/<int:id>/<int:name>")
def show(id,name):
    details ={"student_id":id,"student_name":name}
    return details
```

Url: http://127.0.0.1:5000/details/1001/bhjbhj

Not found: url entered doesnt matches the url mentioned in the route

Q6. Consider the following Python code snippet “code.py”.

[MSQ: 3 points]

Filename: code.py

```
import sys
from jinja2 import Template
vars = sys.argv

course_technologies = {'python': 'backend', 'javascript': 'frontend'}
template = Template("This course focuses on {{ technology }} development.")

if len(vars) > 2 and vars[2] in course_technologies:
    course = vars[1]
    technology = course_technologies[course]
    print(template.render(technology=technology))
else:
    print("Please specify a valid course name!")
```

Which of the following will be the correct command line input to the terminal to get the output : `This course focuses on backend development.` ?

A. `python code.py course python`

B. `python code.py python javascript`

C. `python code.py python python`

D. `python code.py python backend`

Answer: B, C

vars= sys.argv =[code.py, course, python]
vars[0]> code.py, vars[2]= python

A: KeyError: 'course'
B and C> correct output
D: backend is not key, if condition will be false and output will be: `Please specify a valid course name!`

Q7. Consider the following flask application.

[MSQ: 3 points]

app.py

```
from flask import Flask, abort, request

app = Flask(__name__)

@app.route('/validate/<int:number>')
def validate_number(number):
    if number % 2 == 0 and number % 3 == 1:
        abort(400, "Bad Request: Invalid number provided")
    return f'<h1>Valid Number: {number}</h1>'

app.run(debug=True)
```

Which of the following statements is/are true if the application is running locally on http://127.0.0.1:5000 ?

A. For URL `http://127.0.0.1:5000/validate/4` the browser will render
Valid Number: 4

B. For URL `http://127.0.0.1:5000/validate?number=5` the browser will render
Bad Request: Invalid number provided

C. For URL `http://127.0.0.1:5000/validate/4` the browser will render
Bad Request: Invalid number provided

D. For URL `http://127.0.0.1:5000/validate/5` the browser will render
Valid Number: 5

Answer: C, D

Q8. Consider the following flask app and Jinja2 template.

[MCQ: 3 points]

app.py

```
from flask import Flask, render_template
app = Flask(__name__)

@app.route('/')
def index():
  return render_template("index.html", links=['hoME', 'PROfile',
'Contact', 'SITEMAP'])

app.run()
```

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
    <title>Macro</title>
</head>
<body>
    {% macro unordered_list(items)%}
        <ul>
        {% for item in items %}
            <li><a href="/{{item}}">{{item|capitalize}}</a></li>
        {% endfor %}
        </ul>
    {% endmacro %}
    {{ unordered_list(links) }}
</body>
</html>
```

If the flask app is running locally on `http://127.0.0.1:5000`. What will be the output on the browser for the base URL?

A.

- Home
- Profile
- Contact
- Sitemap

B.

- home
- profile
- contact
- sitemap

C.

- HoME
- PROfile
- Contact
- SITEMAP

D.

- HOME
- PROFILE
- CONTACT
- SITEMAP

Answer: A

Python >func, jinja>macro

Q9. Consider the following python code snippet app.py, the html files, base.html and home.html residing in “templates” folder.

[MCQ:3 points]

app.py

```
from flask import Flask, render_template
app = Flask(__name__)
@app.route('/')
def home():
      return render_template('home.html')
app.run(debug=True)
```

home.html

```
{% extends "base.html" %}
{% block content %}
<h3>Welcome to MAD I</h3>
{% endblock %}
```

base.html

```
<!DOCTYPE html>
<html lang="en">
<head>
      <title>IITM</title>
</head>
<body>
      <h1 style="color: violet;"> IITM BS Degree </h1>
      {% block content %}
      {% endblock %}
</body>
</html>
```

What will be the rendered output for base URL if flask app is running locally on http://localhost:5000

A.

Welcome to MAD I

B.

IITM BS Degree

Welcome to MAD I

C.

Welcome to MAD I

IITM BS Degree

D. Error

Answer: B

```
<!-- <!DOCTYPE html>

<html lang="en">

<head>

    <title>IITM</title>

</head>

<body>

    <h1 style="color: violet;"> IITM BS Degree </h1>

    {% block content %}

    <h3>Welcome to MAD I</h3>

    {% endblock %}

</body>

</html> -->
```

Q10. Consider the following flask application.

[MSQ: 4.5 points]

app.py

```
from flask import Flask
app = Flask(__name__)

@app.route('/home/<path:url_path>')
def course(url_path):
    return 'The path is: ' + url_path

@app.route('/home/details/<student_id>/<course_id>')
def home(student_id, course_id):
    return f'The student-id and course-id are {student_id} and {course_id}
respectively.'

@app.route('/home/student/<student_id>/<course_id>')
def details(student_id, course_id):
    details = {'course_id': course_id,'student_id': student_id}
    return details

app.run(debug=True)
```

Which of the following statements is/are true if the application is running locally on `http://127.0.0.1:5000` ?

A. For URL, `http://127.0.0.1:5000/home/M101/CS2003` browser will render; `The student-id and course-id are M101 and CS2003 respectively.`

B. For URL, `http://127.0.0.1:5000/home/student/M101/CS2003` browser will render

```
{
  "course_id": "CS2003",
  "student_id": "M101"
}
```

C. For URL `http://127.0.0.1:5000/home/details/M101/CS2003` browser will render; `The student-id and course-id are M101 and CS2003 respectively.`

D. For URL, `http://127.0.0.1:5000/home/M101/CS2003` browser will render; `The path is: M101/CS2003`

Answer: B, C, D

Q11. Consider the below flask application.

[MCQ : 3 points]

```
from flask_sqlalchemy import SQLAlchemy
from flask import Flask

app = Flask (__name__)
app.config ['SQLALCHEMY_DATABASE_URI'] = 'sqlite:///testdb.sqlite3'
db = SQLAlchemy(app)
app.app_context().push()

class Material(db.Model):
   m_id = db.Column('m_id', db.Integer, primary_key = True)
   name = db.Column('name', db.String(100), unique = True)

db.create_all()
material1 = Material(name = 'Steel')
db.session.add(material1)
material2 = Material(name = 'Iron')
material3 = Material(name = 'Aluminium')
db.session.add(material2)
db.session.commit()
db.session.add(material3)

all_material = Material.query.all()
print([(x.m_id, x.name) for x in all_material])
```

If you run the flask application using a terminal. What will be the output in the terminal?

A. `[(1, 'Steel'), (2, 'Iron')]` will be displayed in the terminal and two records will be added in the “testdb” database.

B. `[(1, 'Steel'), (2, 'Iron'), (3, 'Aluminium')]` will be displayed in the terminal and three records will be added in the “testdb” database.

C. `[(1, 'Steel'), (2, 'Iron'), (3, 'Aluminium')]` will be displayed in the terminal and two records will be added in the “testdb” database.

D. `[(1, 'Steel', 2 , 'Iron', 3 , 'Aluminium')]` will be displayed in the terminal and two records will be added in the “testdb” database.

Answer: C

```
all_material = Material.query.all()> [objects]=[<object1>,<object2>..etc]
Query data> everything which is there in session but the data will only appear in
db when it is committed
```

Q12. Consider the following flask application.

[MSQ: 3 points]

app.py

```
from flask import Flask, render_template, url_for, request, redirect
app = Flask(__name__)
list_of_courses = ['Java', 'Python', 'DBMS', 'PDSA']

@app.route("/if")
def if_loop():
      name = request.args.get("name")
      if name == "MADI":
            return redirect("/home")
      elif name in list_of_courses:
            return url_for("for_loop")
      return "You are not authorized to view this page"

@app.route("/home")
def home_page():
      return "Welcome to MADI!"

@app.route("/for")
def for_loop():
      return render_template("for_course.html", courses=list_of_courses)

app.run(debug=True)
```

Which of the following statements is/are true if the application is running locally on `http://127.0.0.1:5000` ?

A. For URL, `http://127.0.0.1:5000/if?name=MADI` browser will display, `/home` as output.

B. For URL, `http://127.0.0.1:5000/if?name=Java` browser will render, `for_course.html`

C. For URL `http://127.0.0.1:5000/if` browser will display, `You are not authorized to view this page!` as output.

D. For URL `http://127.0.0.1:5000/if?name=Python` browser will display, `/for` as output.

Answer: C, D

1. render_template("file_name",data)> renders the html
2. redirect("endpoint")> redirect to that endpoint and will display the data accordingly
3. url_for("def name")> endpoint
4. redirect(url_for("def name"))> redirect to endpoint

Q13 Consider the following table “emp” created in SQLite database corresponding to model class “Employee” using flask_sqlalchemy.

[MCQ: 3 points]

Id	Name	Designation	Gender	Salary
1	Raji	Headmaster	Female	4500
2	Ram	Teacher	Male	5000
3	Raveena	Teacher	Female	3000
4	Reshma	Technical staff	Female	2000
5	Ravi	Teacher	Male	1000

Which of the following code snippets correctly increases the salary of all the teachers by 1000 Rupees when typed in the Python console?

A.

[object1 , object 2, object3]

```
>>> emp = Employee.query.filter_by(Designation = 'Teacher').all()
>>> for staff in emp:
... staff.Salary += 1000
...
>>> db.session.commit()
```

B.

```
>>> emp = Employee.query.filter_by(Gender = 'Female').all()
>>> staff.Salary += 1000
>>> db.session.commit()
```

C.

```
>>> emp = Employee.query.filter_by(Gender = 'Female').all()
>>> for staff in emp:
... staff.Salary += 1000
...
>>> db.session.commit()
```

D.

```
>>>emp = Employee.query.filter(Employee.Gender.like('T%')).all()
>>> for staff in emp:
... staff.Salary += 1000
...
>>> db.session.commit()
```

Answer: A

The default behavior of .filter() with multiple arguments is already an implicit AND. So these two are equivalent:

```
# Implicit AND
User.query.filter(User.username == 'john', User.email ==
'john@example.com')

# Explicit AND
User.query.filter(and_(User.username == 'john', User.email ==
'john@example.com'))
```

OR condition using filter()

```
from flask_sqlalchemy import SQLAlchemy
from sqlalchemy import or_

db = SQLAlchemy()

class User(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    username = db.Column(db.String(80))
    email = db.Column(db.String(120))

# Query users where username is 'john' OR email is 'john@example.com'
users = User.query.filter(or_( User.username == 'john',User.email == 'john@example.com')).all()
```

AND condition using filter()

```
from flask_sqlalchemy import SQLAlchemy
from sqlalchemy import and_
# Query users where username is 'john' AND email is 'john@example.com'
users=User.query.filter(and_(User.username == 'john',User.email == 'john@example.com'))
```

# Equivalent:(And condition)
User.query.filter_by(username='john', email='john@example.com')
User.query.filter(User.username == 'john', User.email == 'john@example.com')

Conclusion: filter can be used for both AND and OR condition but filter_by can be used only for AND condition.

Here's why:

- `filter_by()` is just a simpler, keyword-arg version of `filter()` meant for straightforward **AND** conditions using **equality only**.
- It **does not** support more complex expressions like `or_()`, `and_()`, comparisons (`>`, `<`, `!=`), etc.

>>LEVELS OF TESTING:

1.MODULE LEVEL>> pytest filename

-here we are giving filename by ourselves, and it checks module within file so no issue,(doesn't matter model name starts with test or not)
-if you give same name to all test cases last will run ,overwrites

```
#for function name
# test_case1 collected
# case2_test ignored
# case_test_3 ignored
```

one function one assert works as return, compiler wont go to second assert, for multiple assert use if else

2.FOLDER LEVEL or directory level>> pytest

all the modules which start with test are tested

```
#for module name
# test_file1.py collected
# file2_test.py collected
# file_test_3.py ignored(but when I give by myself no problem, as it checks within file)
```

pytest my_file.py >>it runs this file as test module
but if test cases does not start with name test they are not collected. While using marker or keyword first all valid cases in module are selected then based on condition deselection

>>KEYWORD BASED SELECTION OF TEST CASES

```
def test_case1():
    assert do_math(6,1)==37

def test_cases_2():
    assert do_math(4,3)==19
```

pytest test_file3.py -k cases

>> MARKERS (defined object in pytest so import)

constraint that cant modify fn name for keyword so use marker,tagging this function to be selected with the help of marker

```
@pytest.mark.my_selected_ones
 def test_case2():
     assert do_math(4,3)==25
```

```
pytest file_test_3.py -m my_selected_ones
pytest test_file3.py -m my_selected_ones --disable-warnings
```

3. CLASS LEVEL TESTING:

if many test cases cant keep on tagging nor can change function name so as far as optimization is concerned speed is considered it is not efficient so class, grouping of multiple test cases into single

>class name used as keyword

```
pytest test_file4.py -k Test_class0
```

BEST LUCK FOR YOUR END SEM EXAM!!!!